1. Analyze and Preprocess data - Check if the dataset has missing values or has any other problem.
2. Feature Engineering
3. Divide the dataset into 2 training and test sets
4. Use logistic model Regression. Try to apply different *solver* and *penalty* to find the best one.
5. Perform model on training set and test set
6. Measure performance of the model.
7. Which metric is your main metric and why? Which solver and penalty have you chosen? (<= 100 words)

How can I measure your point:

1. Your function is callable and runs correctly
2. The performance of your model (in full pipeline) is acceptable. The final error based on my train and test set is low enough.
3. The data preprocessing is correct or make sense
4. The Feature engineering is correct or make sense
5. Any other additional process will be considered a small plus point.

Submission Link: https://forms.gle/M2CxqVGrKLTzqR7g9 (Submit your .ipynb file)

- Age: This is the attribute that describes the age of the patient. There is data type $int64$, the highest value is 29, and the lowest is 77.
- Sex: This is the attribute indicating the gender of the patient, where 0 indicates male patient, 1 female patient.
- ChestPainType: This is the attribute that indicates the patient's chest pain level. With levels 0, 1, 2, and 3.
- RestingBP: This is the attribute that indicates the patient's blood pressure with data type $int64$, the value is in the range [94, 200]
- Cholesterol: This attribute indicates the patient's cholesterol level as measured in the hospital. Has the data type $int64$, where the value is in [126, 564]
- FastingBS: This is an attribute that describes the patient's fasting blood sugar. In which, if the patient has more than 120mg/dl sugar = 1, otherwise = 0.
- RestingECG: This property displays the results of the ECG from 0 to 2 (0, 1, 2). Where each value indicates the severity of the pain.
- thalach: Patient's highest heart rate
- ExerciseAngina: Whether or not you have angina during exercise. Yes denotes 1, no denotes 0.
- Oldpeak: Attribute expressing the stress level of the patient. Has a value of type $float64$, the value is in [0, 6.2]
- ST_Slope: Patient's condition during exercise. Includes [Upsloping, Flat, Down sloping] states that are sequentially digitized to [0, 1, 2].
- ca: number of major vessels (0-4) colored by flourosopy - given
- thal: 0 = normal; 1 = fixed defect; 2 = reversable defect
- HeartDisease: Results of the patient's condition. 1 is for signs of heart disease, 0 is for no signs of heart disease.

Load Dataset

In [32]:
```
import pandas as pd
PATH = r"D:\Introduction-to-Machine-Learning_MTH10353\Homework\2\heart.csv"
df = pd.read_csv(PATH)
df
```

Out[32]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	46	1	0	120	249	0	0	144	0	0.8	2	0	3	0
1	71	0	1	160	302	0	1	162	0	0.4	2	2	2	1
2	71	0	1	160	303	0	1	163	0	0.4	2	2	2	1
3	71	0	1	160	304	0	1	164	0	0.4	2	2	2	1
4	71	0	1	160	305	0	1	165	0	0.4	2	2	2	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
881	68	0	2	120	211	0	0	115	0	1.5	1	0	2	1
882	44	0	2	108	141	0	1	175	0	0.6	1	0	2	1
883	52	1	0	128	255	0	1	161	1	0.0	2	1	3	0
884	59	1	3	160	273	0	0	125	0	0.0	2	0	2	0
885	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

886 rows × 14 columns

Data Analysis

In [33]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 886 entries, 0 to 885
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       886 non-null    int64
 1   sex       886 non-null    int64
 2   cp        886 non-null    int64
 3   trestbps  886 non-null    int64
 4   chol      886 non-null    int64
 5   fbs       886 non-null    int64
 6   restecg   886 non-null    int64
 7   thalach   886 non-null    int64
 8   exang     886 non-null    int64
 9   oldpeak   886 non-null    float64
 10  slope     886 non-null    int64
 11  ca        886 non-null    int64
 12  thal      886 non-null    int64
 13  target    886 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 97.0 KB
```

In [34]:
```
df.describe()
```

Out[34]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak
count	886.000000	886.000000	886.000000	886.000000	886.000000	886.000000	886.000000	886.000000	886.000000	886.000000
mean	54.007901	0.683973	1.005643	132.117381	247.497743	0.168172	0.495485	149.492099	0.310384	1.083296
std	9.126292	0.465186	1.024542	16.807511	47.209995	0.374230	0.526669	23.921327	0.462912	1.147935
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000
25%	47.000000	0.000000	0.000000	120.000000	212.000000	0.000000	0.000000	133.000000	0.000000	0.000000
50%	54.000000	1.000000	1.000000	130.000000	245.000000	0.000000	0.000000	153.000000	0.000000	0.800000
75%	60.000000	1.000000	2.000000	140.000000	274.000000	0.000000	1.000000	166.000000	1.000000	1.800000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	206.000000	1.000000	6.200000

In [35]:
```
# Data analysis
# 1. Analyze and Preprocess data - Check if the dataset has missing values or has any other problem.

print("Missing values:")
df.isnull().sum()
```

```
Missing values:
```

Out[35]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

There are no missing values in the dataset, and all values are valid. Therefore, we check invalid values next.

In [36]:
```
print("Invalid values:")
print("Age: ", df[df["age"] < 0])
print("RestingBP: ", df[(df["trestbps"] < 94) | (df["trestbps"] > 200)])
print("Cholesterol: ", df[(df["chol"] < 126) | (df["chol"] > 564)])
print("Oldpeak: ", df[(df["oldpeak"] < 0) | (df["oldpeak"] > 6.2)])
print("thal: ", df[~df["thal"].isin([0, 1, 2, 3])])
print("ca: ", df[~df["ca"].isin([0, 1, 2, 3, 4])])
print("ST_Slope: ", df[~df["slope"].isin([0, 1, 2])])
print("ChestPainType: ", df[~df["cp"].isin([0, 1, 2, 3])])
print("ExerciseAngina: ", df[~df["exang"].isin([0, 1])])
print("FastingBS: ", df[~df["fbs"].isin([0, 1])])
print("RestingECG: ", df[~df["restecg"].isin([0, 1, 2])])
print("HeartDisease: ", df[~df["target"].isin([0, 1])])
```

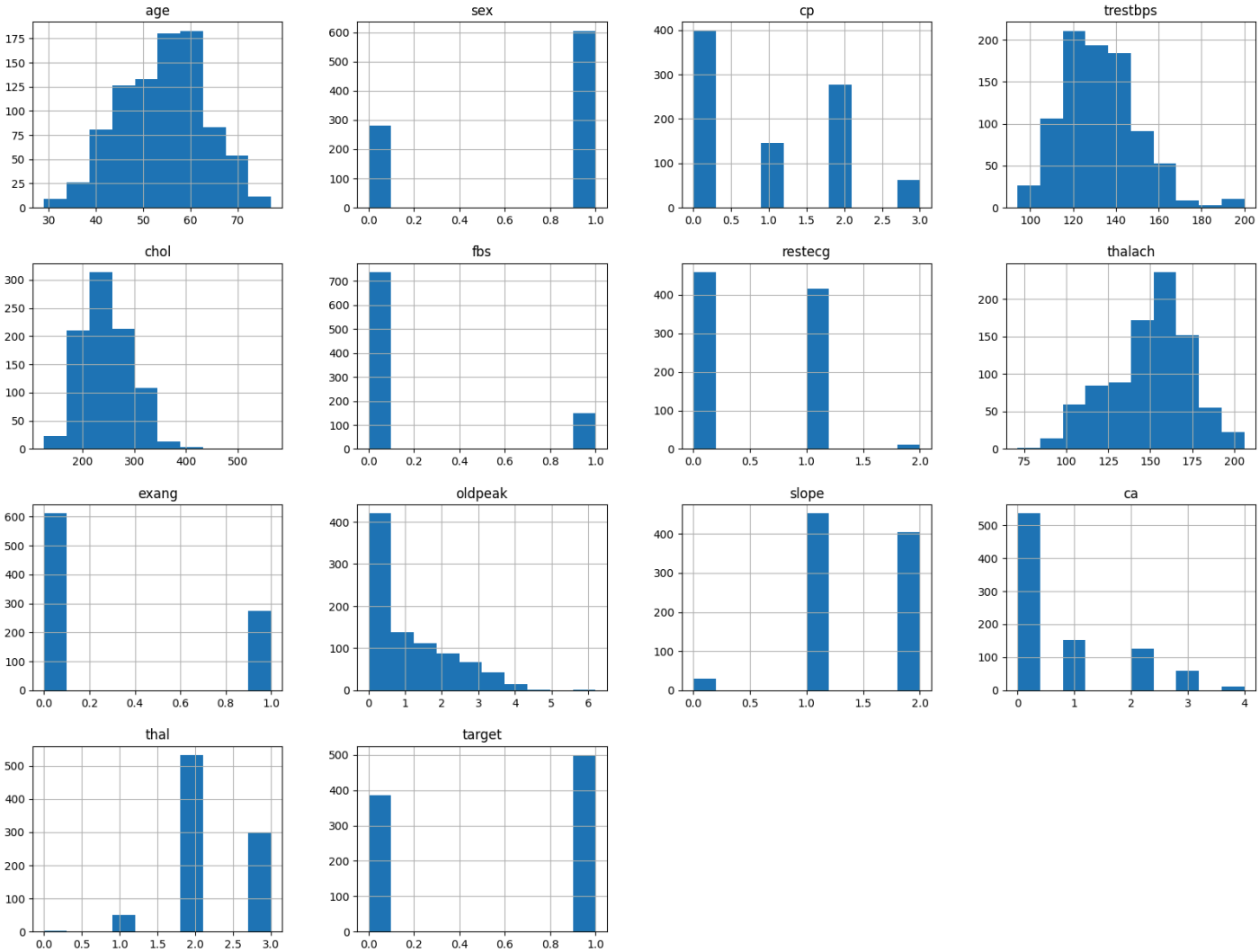
```
Invalid values:
Age:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
RestingBP:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
Cholesterol:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
Oldpeak:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
thal:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
ca:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
ST_Slope:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
ChestPainType:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
ExerciseAngina:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
FastingBS:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
RestingECG:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
HeartDisease:  Empty DataFrame
Columns: [age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target]
Index: []
```

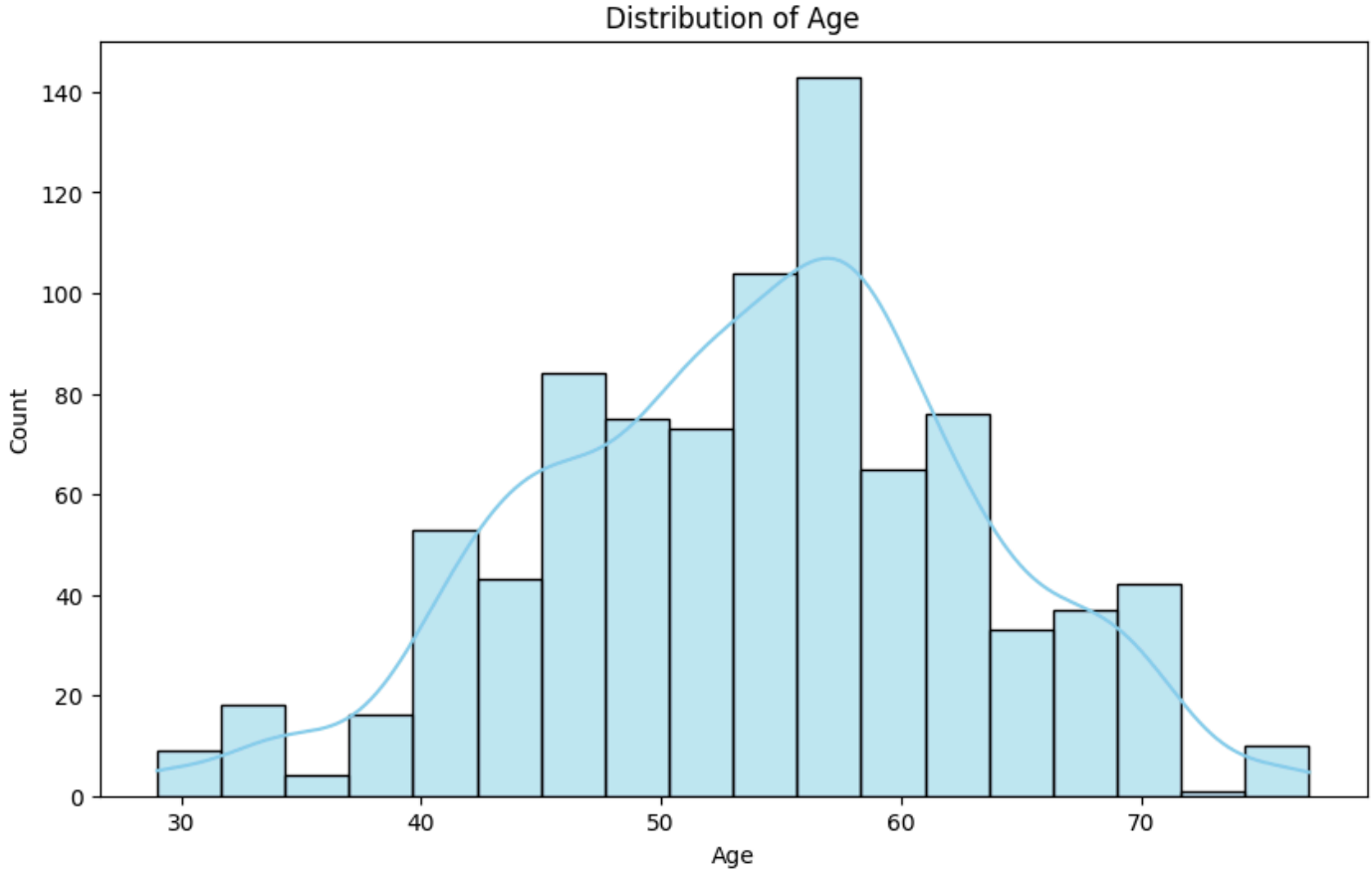
There are no invalid values in the dataset, and all values are valid. Therefore, we will not need to handle invalid values.

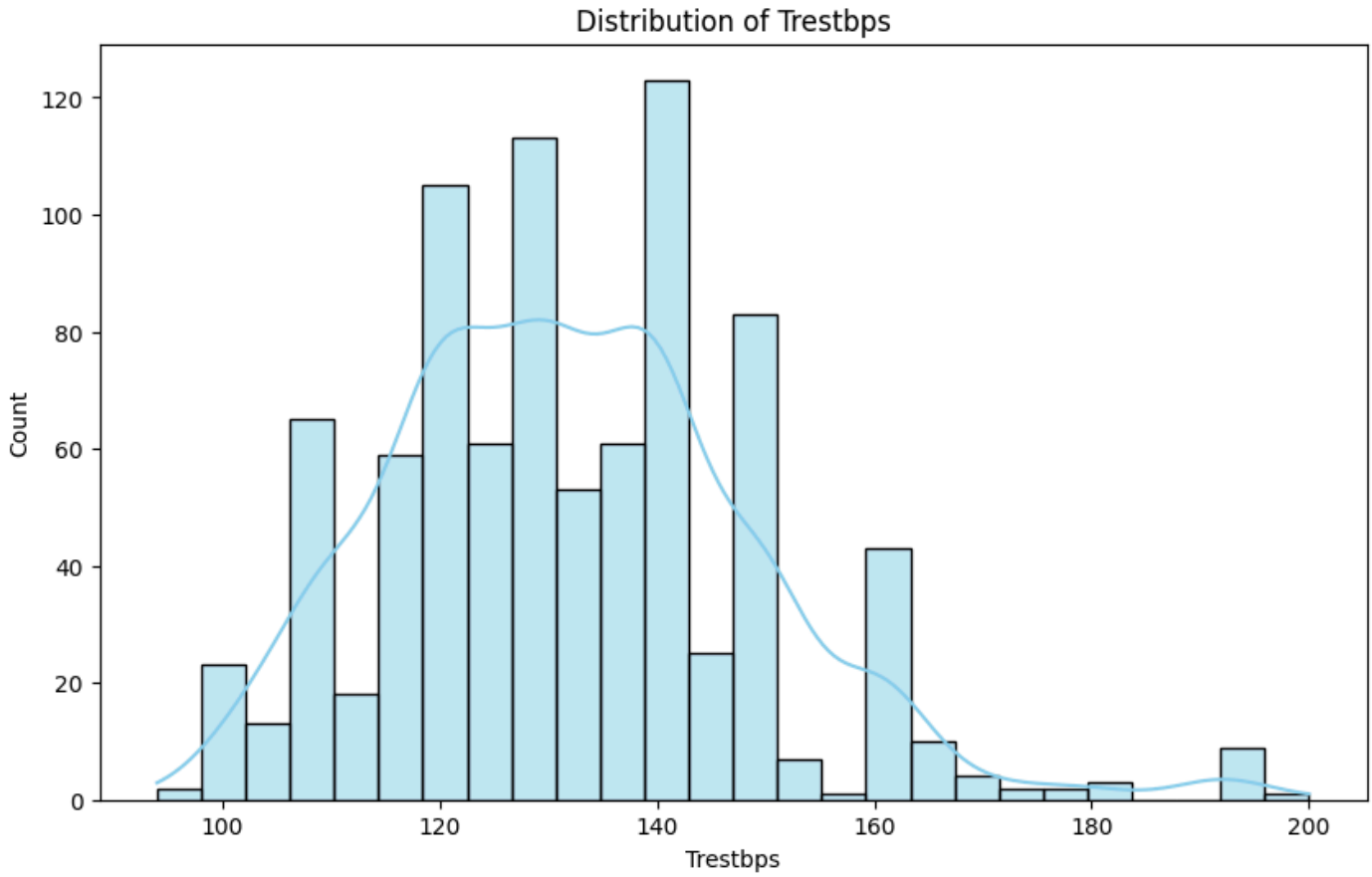
Note: ở cột thal thì theo nguồn https://ieee-dataport.org/documents/heartcsv thì giá trị là [0, 1, 2, 3] chứ không phải [0, 1, 2] như đã đề cập ở đề bài.

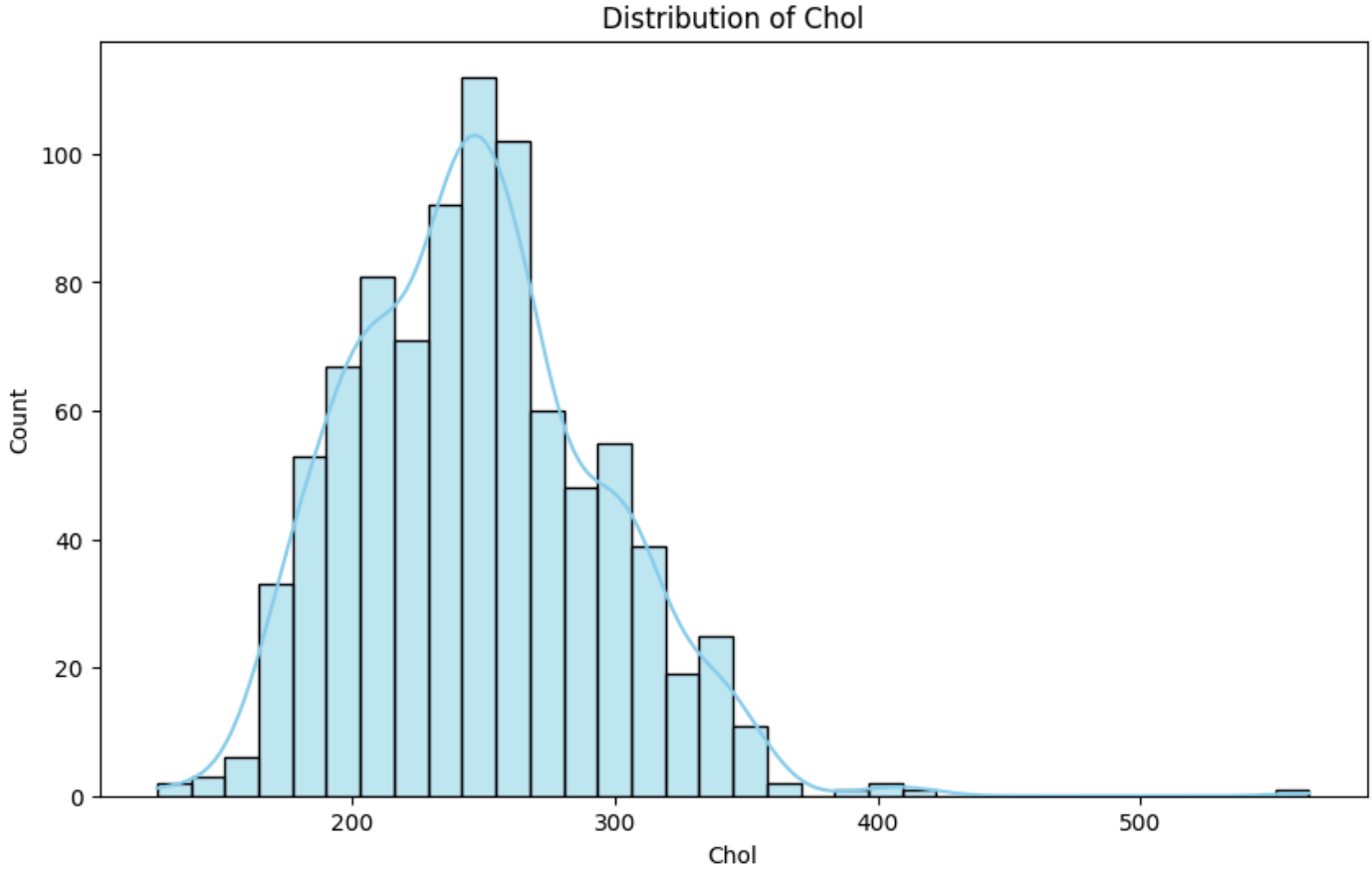
In [37]:
```
from matplotlib import pyplot as plt
import seaborn as sns

Columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'th
```

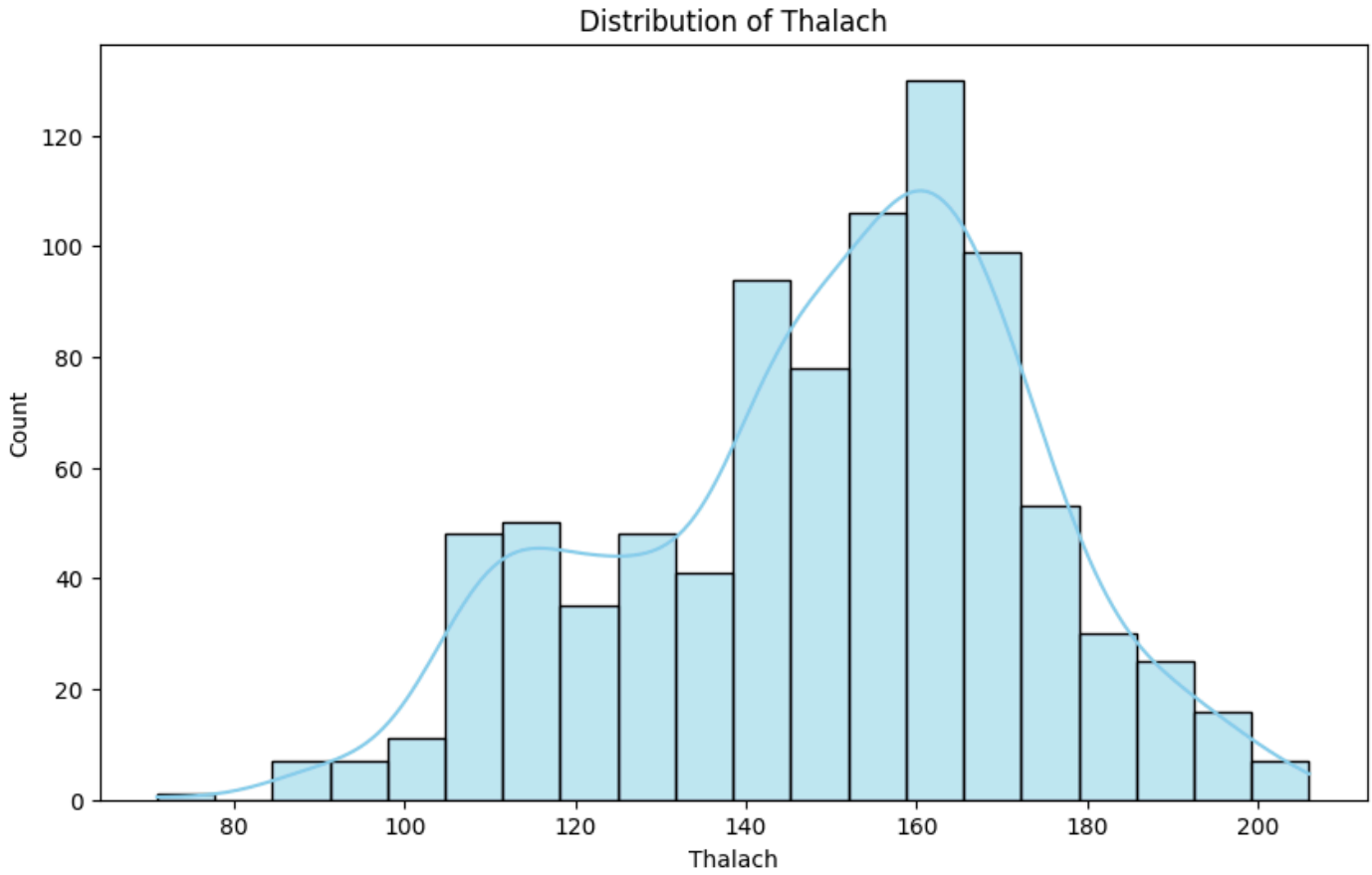
```python
df.hist(column=Columns, figsize=(20, 15))
non_binary_columns = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
for column in non_binary_columns:
    plt.figure(figsize=(10, 6))
    sns.histplot(df[column], kde=True, color='skyblue')
    plt.title(f'Distribution of {column.capitalize()}')
    plt.xlabel(column.capitalize())
    plt.ylabel('Count')
    plt.show()
binary_columns = ['sex', 'fbs', 'exang', 'target']
for column in binary_columns:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=df[column])
    plt.title(f'Count of {column.capitalize()}')
    plt.xlabel(column.capitalize())
    plt.ylabel('Count')
    plt.show()
```

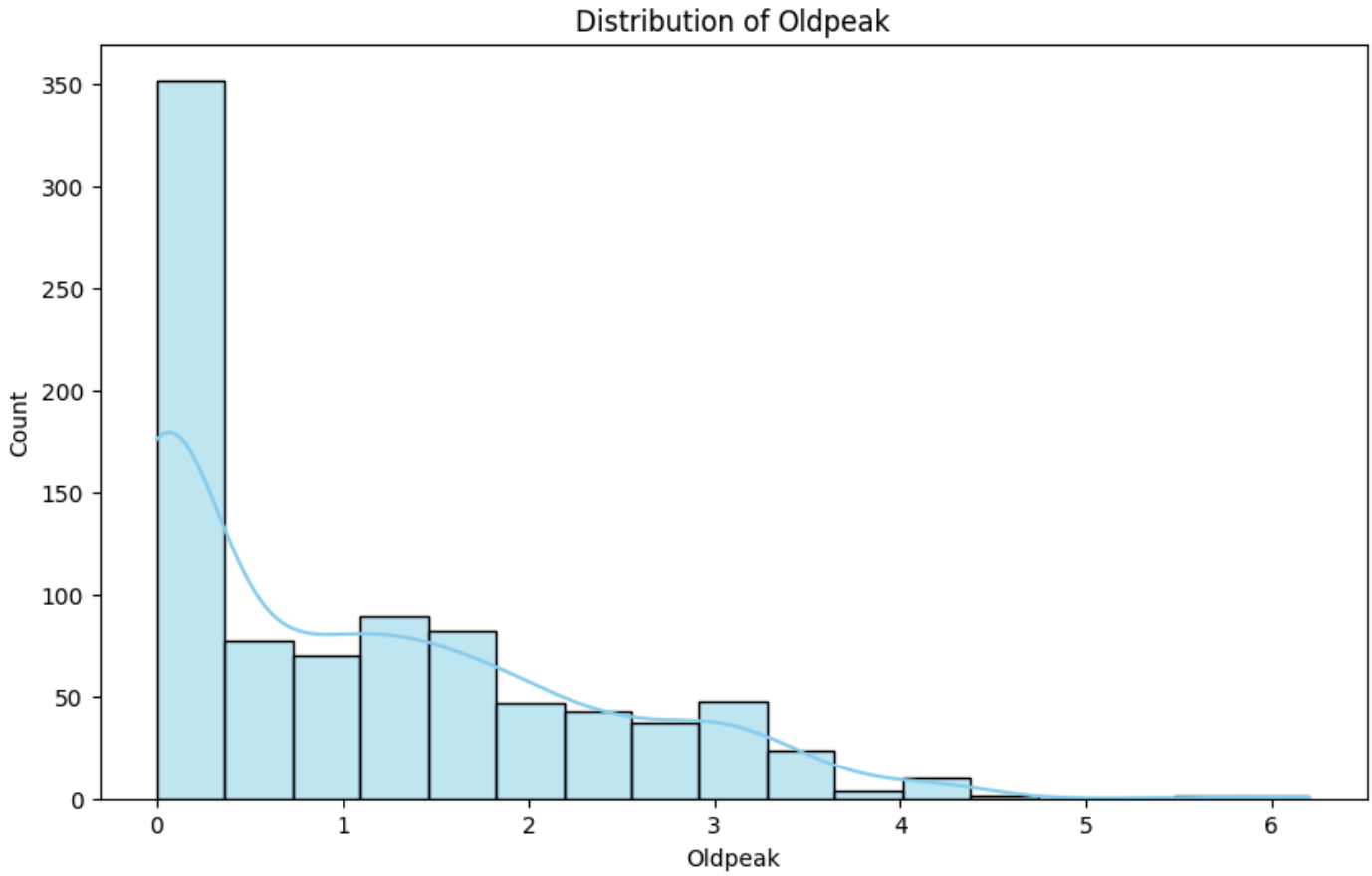


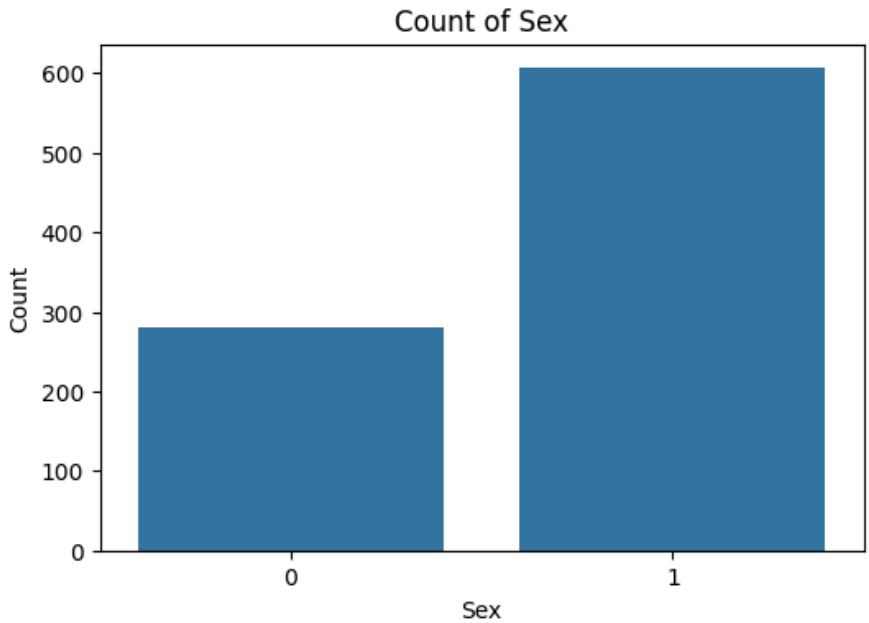

Distribution of Age
Count
Age
0
20
40
60
80
100
120
140
30
40
50
60
70

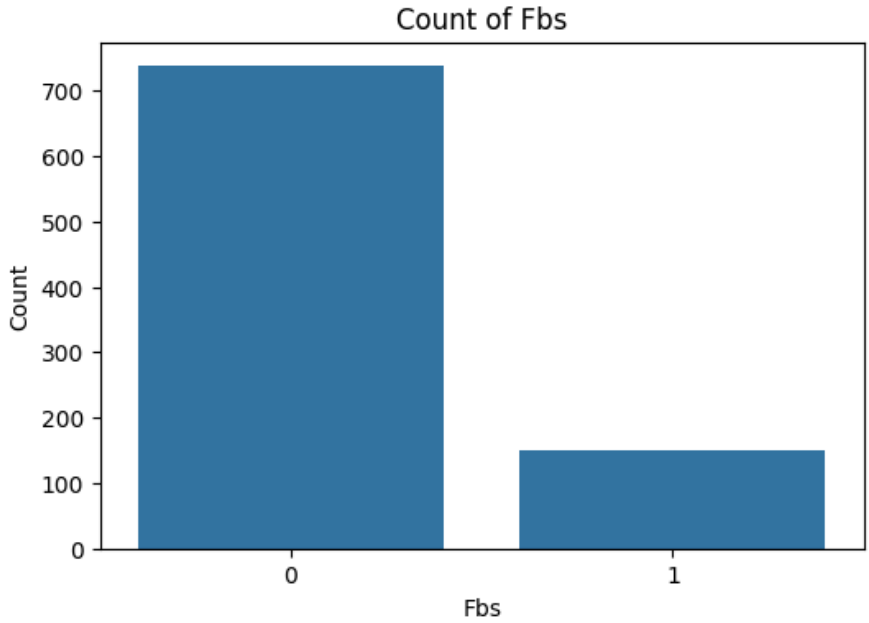

Distribution of Trestbps
Count
Trestbps
0
20
40
60
80
100
120
100
120
140
160
180
200

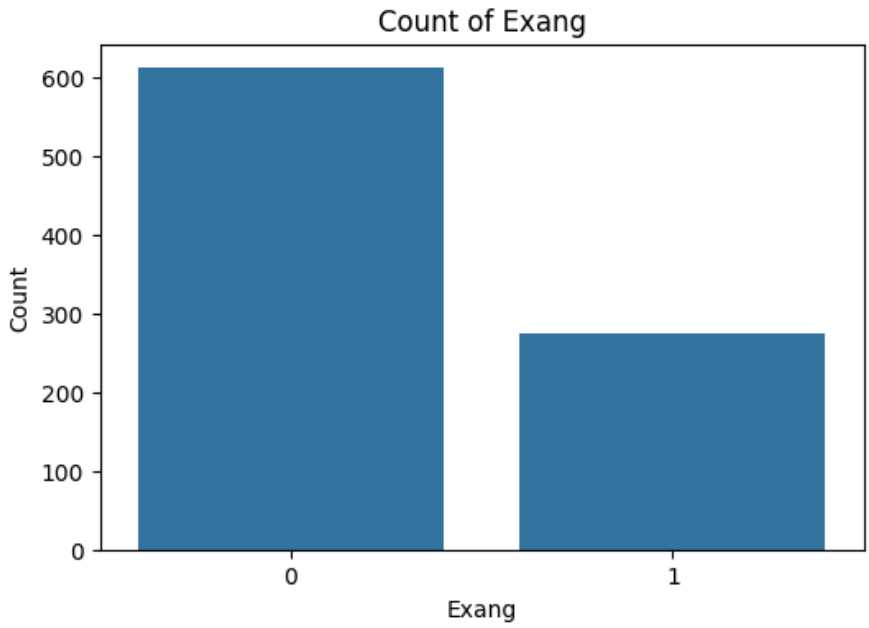

Distribution of Chol
Count
Chol
0
20
40
60
80
100
200
300
400
500


Distribution of Thalach
Count
Thalach
0
20
40
60
80
100
120
80
100
120
140
160
180
200


Distribution of Oldpeak
Count
Oldpeak
0
50
100
150
200
250
300
350
0
1
2
3
4
5
6

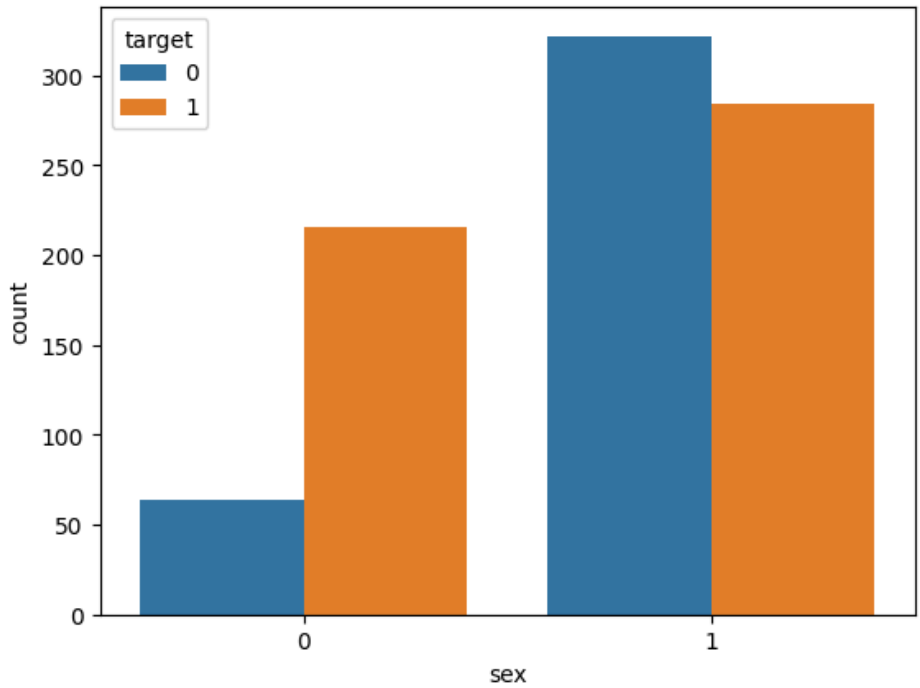

Count of Sex
Count
Sex
0
100
200
300
400
500
600
0
1


Count of Fbs
Count
Fbs
0
100
200
300
400
500
600
700
0
1



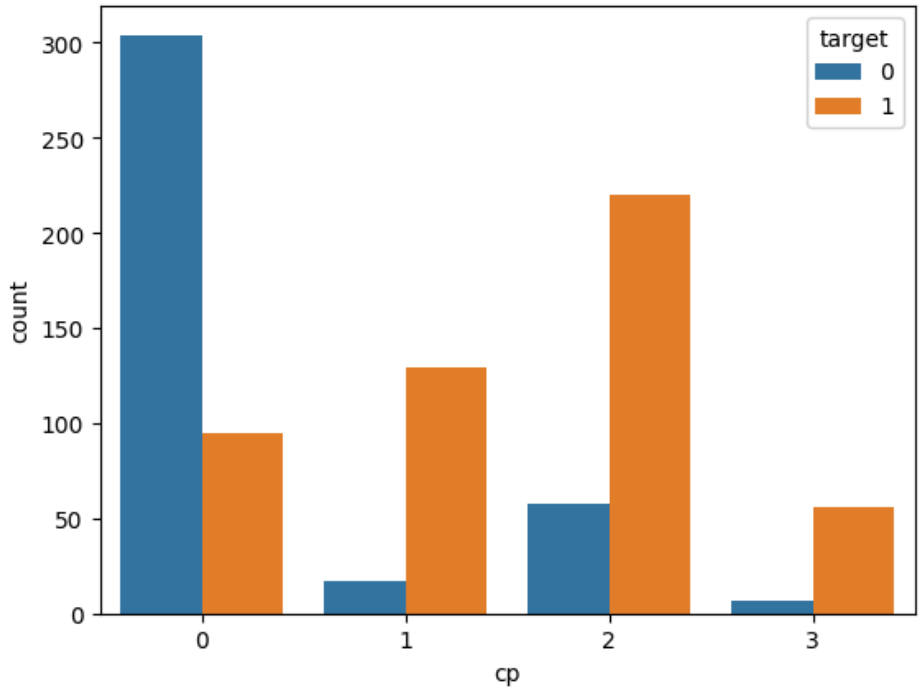
In [38]:
```
sns.countplot(data=df, x='sex', hue='target')
```

Out[38]:
```
<Axes: xlabel='sex', ylabel='count'>
```



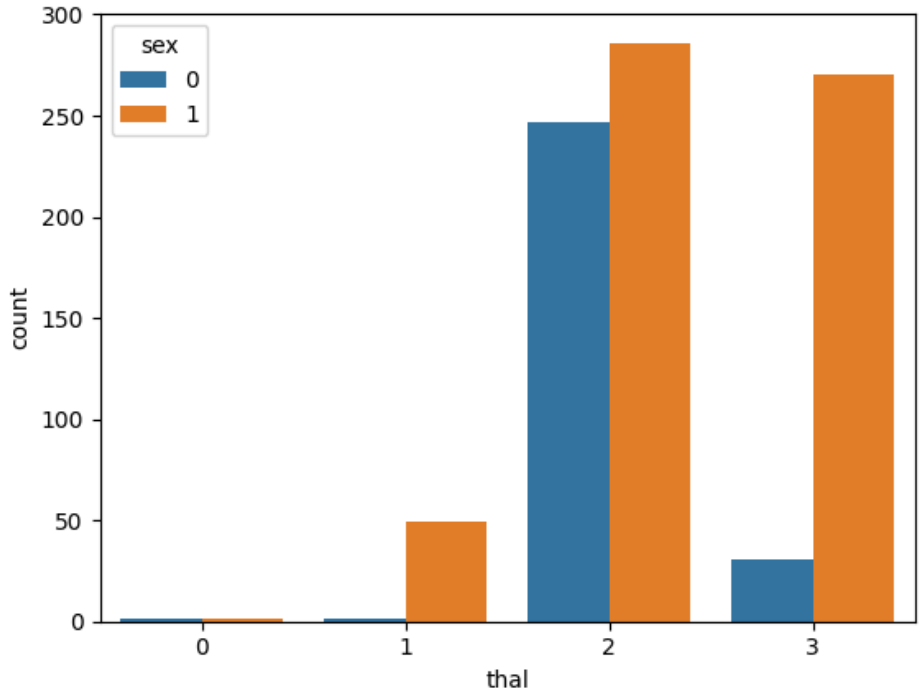
In [39]:
```
sns.countplot(data=df, x='cp', hue='target')
```

Out[39]: <Axes: xlabel='cp', ylabel='count'>



In [40]:
```
sns.countplot(data=df, x='thal', hue='sex')
```

Out[40]: <Axes: xlabel='thal', ylabel='count'>



Preprocessing

Because the dataset does not contain any missing values, we will not need to handle missing values. However, still have to dropna() in preprocessing pipeline to make sure that there is no missing value in the future when we update the dataset.

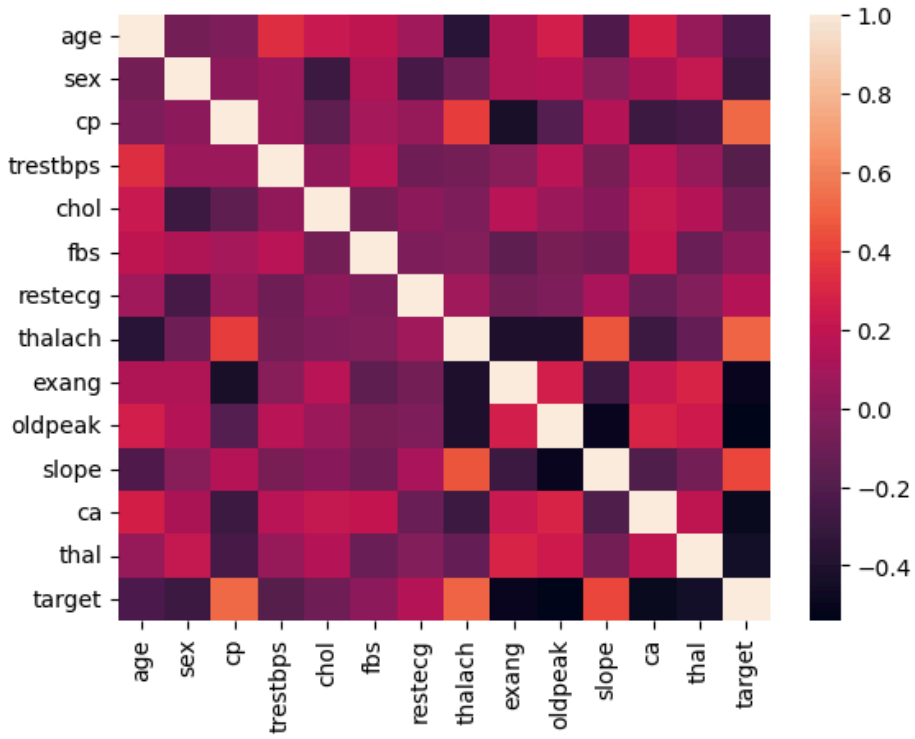
In [41]:
```
def preprocessing_data(df):
    """
    Preprocess your data (eg. Drop null datapoints or fill missing data)
    :param df: pandas DataFrame
    :return: pandas DataFrame
    """
    # Todo: preprocess data
    df = df.dropna()  # Drop rows with missing values (if any)
    return df
```

In [42]:
```
df = preprocessing_data(df.copy())
```

Feature Engineering

In [43]:
```
# Heatmap
import seaborn as sns

def apply_feature_engineering(df):
    """
    Apply all feature engineering to transform your data into number
    :param df: pandas DataFrame
    :return: pandas DataFrame
    """
    # Todo: Normalize your data
    df = (df - df.min()) / (df.max() - df.min())
    return df

df = apply_feature_engineering(df)
sns.heatmap(df.corr()) # Show heatmap after normalized data
```

Out[43]: <Axes: >



In [44]:
```
def prepare_X_y(df):
    """
    Feature engineering and create X and y
    :param df: pandas dataframe
    :return: (X, y) output feature matrix (dataframe), target (series)
    """
    # Todo: Split data into X and y (using sklearn train_test_split). Return two dataframes
    X = df.drop(columns=['target'])  # Drop the target column to get features
    y = df['target']  # Select the target column
    return X, y

X, y = prepare_X_y(df)
```

Apply machine learning model

Train-test split

I choose `TRAIN_SIZE = 0.7` because it provides a good balance between having enough data for training the model and having enough data for testing its performance.

With 70% of the data used for training, the model can learn effectively from a substantial portion of the dataset, while the remaining 30% allows for a robust evaluation of the model's generalization to unseen data.

In [45]:
```
from sklearn.model_selection import train_test_split
RANDOM_STATE = 69 # funny number lol
TRAIN_SIZE = 0.7

trainX, testX ,trainY, testY = train_test_split(X, y, train_size=TRAIN_SIZE, random_state=RANDOM_STATE)
```

Build SK-learn model

In [46]:
```
from sklearn.metrics import classification_report
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, accuracy_score, f1_score, mean_absolute_error
def build_model(X, y):
    """
    Design your model and train it (including your best params)
    :param X: feature matrix
    :param y: target
    :return: a model
    """

    # Todo: Input your scaler and logistic model into pipeline
    model = make_pipeline(StandardScaler(), LogisticRegression(fit_intercept=True, C=1.0, max_iter=10000, random_state
    # Todo: fit your model with X, y
    model.fit(X, y)

    return model

def calculate_performance(y_true, y_pred):
    """

    :param y_true: ground truth values
    :param y_pred: predictions
    :return:
    """
    # Todo: return your error value like accuracy, f1score, ...
    print("precision", precision_score(y_true, y_pred))
    print("recall", recall_score(y_true, y_pred))
    print("accuracy", accuracy_score(y_true, y_pred))
    print("F1", f1_score(y_true, y_pred))

    # Todo: Only choose one of them as your score for the question 7
    main_score = f1_score(y_true, y_pred)
    return main_score

model = build_model(trainX, trainY)
# Compare on training dataset
pred = model.predict(testX)
calculate_performance(testY, pred)
```

```
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
```

Out[46]:
```
0.9573770491803278
```

In [47]:
```
MAE = []

pred = model.predict(trainX)
print("MAE on train set: ", mean_absolute_error(y_pred=pred, y_true=trainY))
MAE.append(mean_absolute_error(y_pred=pred, y_true=trainY))
pred = model.predict(testX)
print("MAE on test set: ", mean_absolute_error(y_pred=pred, y_true=testY))
MAE.append(mean_absolute_error(y_pred=pred, y_true=testY))
```

```
MAE on train set:  0.04838709677419355
MAE on test set:  0.04887218045112782
```

Extra: I added cross-validation to evaluate model's performance.

In [48]:
```
# Cross validation
def cross_validation(model, X, y, cv=5):
    """
    Perform cross validation and return the average score
    :param model: your model
    :param X: feature matrix
    :param y: target
    :param cv: number of folds
    :return: average score
    """
    from sklearn.model_selection import cross_val_score
    scores = cross_val_score(model, X, y, cv=cv, scoring='f1')
    return scores.mean()
```

```
cv_score = cross_validation(model, X, y)
print("Cross validation score: ", cv_score)
```

```
Cross validation score:  0.921727600895033
```

Modified model for different penalty and solver

In [49]:
```
def build_model_modified(X, y, solver, penalty):
    """
    Design your model and train it (including your best params)
    :param X: feature matrix
    :param y: target
    :param solver: solver for logistic regression
    :param penalty: regularization penalty for logistic regression ('l1', 'l2', or None)
    :return: a model
    """
    custom_l1_ratio = None
    custom_c = 1.0

    if penalty == 'l1':
        custom_l1_ratio = 1.0
    elif penalty == 'l2':
        custom_l1_ratio = 0.0
    elif penalty == 'elasticnet':
        custom_l1_ratio = 0.5
    else:
        custom_l1_ratio = 0.0

    model = make_pipeline(
        StandardScaler(),
        LogisticRegression(
            fit_intercept=True,
            C=custom_c,
            max_iter=10000,
            random_state=RANDOM_STATE,
            solver=solver,
            l1_ratio=custom_l1_ratio,
            # penalty=penalty deprecated in newer versions of scikit-learn, so we use l1_ratio instead
        )
    )

    model.fit(X, y)
    return model
```

Penalty: L1

In [50]:
```
solvers = ['liblinear', 'saga']
for s in solvers:
    model_l1 = build_model_modified(trainX, trainY, solver=s, penalty='l1')
    pred_l1_test = model_l1.predict(testX)
    pred_l1_train = model_l1.predict(trainX)
    print(f"Performance for solver {s}:")
    calculate_performance(testY, pred_l1_test)
    cross_val_score_modified = cross_validation(model_l1, X, y)
    print(f"Cross validation score of modified model with solver {s}: ", cross_val_score_modified)

    MAE = []
    print(f"MAE for solver {s}:")
    print("     -MAE on train set: ", mean_absolute_error(y_pred=pred_l1_train, y_true=trainY))
    MAE.append(mean_absolute_error(y_pred=pred_l1_train, y_true=trainY))
    print("     -MAE on test set: ", mean_absolute_error(y_pred=pred_l1_test, y_true=testY))
    MAE.append(mean_absolute_error(y_pred=pred_l1_test, y_true=testY))
    print(f"     -Difference in MAE for solver {s}: ", MAE[1] - MAE[0])
```

```
Performance for solver liblinear:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver liblinear:  0.9204016331722027
MAE for solver liblinear:
     -MAE on train set:  0.05
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver liblinear:  -0.001127819548872186
Performance for solver saga:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver saga:  0.9208820421921201
MAE for solver saga:
     -MAE on train set:  0.05
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver saga:  -0.001127819548872186
```

Penalty: L2

In [51]:
```
solvers = ['newton-cg', 'lbfgs', 'sag', 'saga', 'liblinear']
for s in solvers:
    model_l2 = build_model_modified(trainX, trainY, solver=s, penalty='l2')
    pred_l2_test = model_l2.predict(testX)
    pred_l2_train = model_l2.predict(trainX)
    print(f"Performance for solver {s}:")
    calculate_performance(testY, pred_l2_test)
    cross_val_score_modified = cross_validation(model_l2, X, y)
    print(f"Cross validation score of modified model with solver {s}: ", cross_val_score_modified)

    MAE = []
    print(f"MAE for solver {s}:")
    print("     -MAE on train set: ", mean_absolute_error(y_pred=pred_l2_train, y_true=trainY))
    MAE.append(mean_absolute_error(y_pred=pred_l2_train, y_true=trainY))
    print("     -MAE on test set: ", mean_absolute_error(y_pred=pred_l2_test, y_true=testY))
    MAE.append(mean_absolute_error(y_pred=pred_l2_test, y_true=testY))
    print(f"     -Difference in MAE for solver {s}: ", MAE[1] - MAE[0])
```

```
Performance for solver newton-cg:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver newton-cg:  0.921727600895033
MAE for solver newton-cg:
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver newton-cg:  0.0004850836769342698
Performance for solver lbfgs:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver lbfgs:  0.921727600895033
MAE for solver lbfgs:
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver lbfgs:  0.0004850836769342698
Performance for solver sag:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver sag:  0.921727600895033
MAE for solver sag:
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver sag:  0.0004850836769342698
Performance for solver saga:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver saga:  0.921727600895033
MAE for solver saga:
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver saga:  0.0004850836769342698
Performance for solver liblinear:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of modified model with solver liblinear:  0.9206034834665677
MAE for solver liblinear:
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for solver liblinear:  0.0004850836769342698
```

Penalty: ElasticNet

In [52]:
```
model_elasticnet = build_model_modified(trainX, trainY, solver='saga', penalty='elasticnet')
pred_elasticnet = model_elasticnet.predict(testX)
print(f"Performance for solver elasticnet:")
calculate_performance(testY, pred_elasticnet)
cross_val_score_elasticnet = cross_validation(model_elasticnet, X, y)
print("Cross validation score of elasticnet model: ", cross_val_score_elasticnet)

MAE = []
pred = model_elasticnet.predict(trainX)
print("     -MAE on train set: ", mean_absolute_error(y_pred=pred, y_true=trainY))
MAE.append(mean_absolute_error(y_pred=pred, y_true=trainY))
pred = model_elasticnet.predict(testX)
print("     -MAE on test set: ", mean_absolute_error(y_pred=pred, y_true=testY))
MAE.append(mean_absolute_error(y_pred=pred, y_true=testY))
print("     -Difference in MAE for elasticnet model: ", MAE[1] - MAE[0])
```

```
Performance for solver elasticnet:
precision 0.9358974358974359
recall 0.9798657718120806
accuracy 0.9511278195488722
F1 0.9573770491803278
Cross validation score of elasticnet model:  0.9208820421921201
     -MAE on train set:  0.04838709677419355
     -MAE on test set:  0.04887218045112782
     -Difference in MAE for elasticnet model:  0.0004850836769342698
```

In [53]:
```
def get_conclusion():
    return (
        "7. My main metric is the F1 score because it perfectly balances precision and recall. " \
        "I chose the L2 penalty because it got the highest cross-validation score and the smallest train-test MAE diff
        "Finally, since solvers got identical metrics, I chose the 'saga' solver because it is fast for large datasets
    )
```

In [54]:
```
get_conclusion()
```

Out[54]:
```
"7. My main metric is the F1 score because it perfectly balances precision and recall. I chose the L2 penalty because
it got the highest cross-validation score and the smallest train-test MAE differenceFinally, since solvers got identi
cal metrics, I chose the 'saga' solver because it is fast for large datasets and flexibly supports all penalty type
s."
```